InsightOps AI: A Governed Multi-Agent ChatBI Platform for Enterprise Decision Intelligence

Haoyuan Shan
Master's in Software Engineering
San Jose State University
San Jose, United States
shanhaoyuan820712@gmail.com

Abstract—Most enterprise business-intelligence systems are either static dashboards or analyst-driven SQL workflows, which leaves business users waiting on a data team for every follow-up question. Asking a large language model to write and run SQL directly does not solve this: an ungoverned model can reference the wrong table, leak sensitive fields, or simply hallucinate a plausible but wrong answer. In this project we built InsightOps AI, a governed multi-agent ChatBI platform that treats a natural-language business question as a workflow, not a single model call. A router classifies the question, a semantic layer constrains what the model may reference, a non-LLM SQL guardrail validates and rewrites every generated SQL candidate before it is allowed to reach a database-level read-only role, a hybrid retrieval pipeline (BM25 keyword scoring fused with real embeddings and an optional cross-encoder rerank stage, backed by pgvector) supplies cited evidence for explanatory questions, and an answer-verification step checks the assembled response against its sources. The platform includes real authentication, role-based access control, and tenant isolation; an LLM-provider gateway with task-based model routing and a deterministic mock provider for cost-free testing; an evaluation runner and release gate that a human can accept only after automated tests, static type checks, and quality thresholds already pass; and an observability stack with a durable, opt-in storage backend that feeds a golden-dataset mining pipeline for continuous retrieval-quality improvement. The system runs locally with Docker Compose and was deployed to a live GKE Autopilot staging cluster, where it was benchmarked under steady-state load, burst load, and deliberate overload (12,000 requests at a 20 RPS target, of which the rate limiter correctly rejected 90% with HTTP 429 rather than letting the system degrade uncontrolled), survived a live pod-kill recovery drill in 21 seconds, and showed that its governed retrieval path uses 98% fewer tokens than a naive full-context baseline on realistic multi-evidence analytical questions. This report summarizes the requirements, design, implementation, security model, staging deployment results, and evaluation results, and discusses the gaps that remain before this is a fully production-hardened cloud service.

Index Terms—multi-agent systems, natural language to SQL, retrieval-augmented generation, SQL guardrail, hybrid retrieval, enterprise business intelligence, evaluation gates, role-based access control

I. Introduction

In most organizations, a business user who wants to know why revenue dropped last month, which segment caused an anomaly, or whether next quarter's revenue can be forecast, still has to route the question to a data analyst and wait. Static dashboards answer only the questions their authors anticipated, and analyst-driven SQL does not scale to every follow-up question a stakeholder might ask.

A natural response is to let a large language model answer directly from a database. This is where most "ChatBI" demos stop, and it is also where they become unsafe: a model with direct database access can be prompted into referencing tables it should not see, can hallucinate a syntactically valid but semantically wrong query, and offers no audit trail explaining why a given answer was produced. Treating the model as the system boundary conflates two very different concerns – understanding a question and being trusted with data access.

InsightOps AI is built around the opposite principle: the language model is a capability inside the system, not the boundary of the system. Every question is decomposed into a workflow – semantic interpretation, SQL generation, guardrail validation, read-only execution, evidence retrieval, analytics, and verification – and every step of that workflow is traced, audited, and gated by an evaluation process before it is trusted as a release-worthy behavior. Permissions, SQL safety, tenant isolation, and observability are treated as first-class system properties, not after-the-fact additions.

The project's scope covers multi-agent orchestration, a semantic layer and governed NL2SQL, a defense-in-depth SQL guardrail, hybrid RAG retrieval with a golden-dataset evaluation loop, an LLM provider gateway, real authentication/RBAC/tenant isolation, an evaluation-and-release-gate pipeline, observability with durable opt-in storage, and both local (Docker Compose) and staging-benchmarked Kubernetes deployment on GKE. It does not yet claim full production-cloud hardening: distributed resilience patterns such as circuit breakers and dead-letter queues, a second real LLM/embedding provider beyond OpenAI, large-scale synthetic load data, and a fully hardened managed-cloud production profile are documented as future work rather than claimed as done.

II. Requirements and Use Cases

InsightOps AI is designed around four personas: a business user who wants an answer without writing SQL, an analyst who wants to reduce repetitive reporting work,

a data/platform team that needs guardrails and auditability, and an admin who needs system health, security, and quality control behind permissioned endpoints.

A. Functional Requirements

1) Accept a natural-language question, authenticate the caller, and resolve their organization/tenant context.
2) Classify the question into a task type (SQL query, chart, analytics, RAG explanation, verification, or file-scoped data) and build an execution plan.
3) Generate a SQL candidate through a semantic layer, then validate, rewrite, and allow/deny it through a non-LLM guardrail before any database access.
4) Execute guardrail-approved SQL only against a database-level read-only role, and enrich the result with charts and analytics/forecasting when relevant.
5) Retrieve cited evidence from business documents through hybrid (keyword + vector) search for explanatory ("why"/"explain") questions, and verify the assembled answer against its sources before returning it.
6) Provide sign-up/sign-in, short-lived access tokens, refresh sessions, and permission-scoped admin endpoints for traces, evaluations, release-gate status, and audit logs.
7) Support user file upload, ownership, and sharing, scoped to the uploading user's organization.
8) Run reproducibly in a local Docker Compose environment and on a Kubernetes deployment, staging-benchmarked on GKE.

B. Non-Functional Requirements

The primary non-functional requirements were governance (no SQL reaches the database without passing a guardrail decision, and no RAG answer is presented without either evidence or an explicit missing-evidence warning), auditability (every guardrail decision, role change, and request is traceable by a single trace_id), testability (core agent and guardrail behavior must be verifiable without a live model, so it can run deterministically in CI), and tenant isolation (query history, documents, evaluation results, and traces must not cross organization boundaries).

C. Use Cases

- A business user asks "what was our highest-revenue month?" and receives a table-backed answer with a chart, without writing SQL.
- An analyst asks "why did support tickets spike in Q2?" and receives an answer grounded in cited document evidence, or an explicit statement that no supporting evidence was found.
- A platform engineer reviews the guardrail audit log to confirm why a particular SQL candidate was denied.

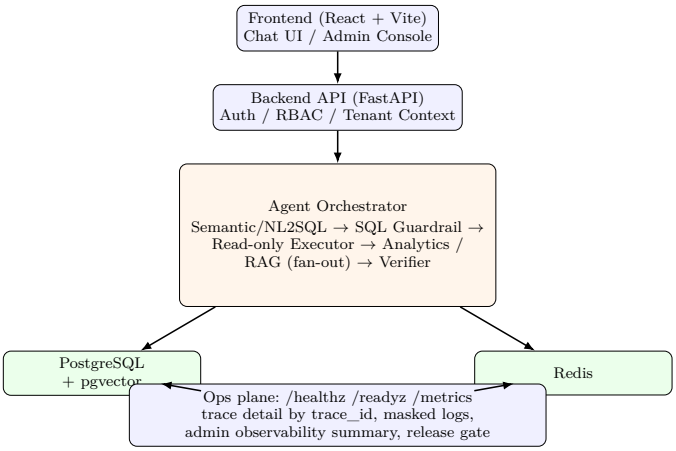

Fig. 1. InsightOps AI high-level architecture: request path from the frontend through auth, orchestration, and the data/operations plane.

- An admin reviews the release-gate dashboard before approving a deployment, and reviews the role-change audit log after granting a new permission.
- The platform mines real production questions into golden-dataset candidates to measure whether retrieval quality is actually improving over time, rather than relying on a one-time fixture.

III. System Design

A. Architecture

Figure 1 shows the system in four layers: a React/Vite chat and admin frontend; a FastAPI backend that resolves authentication and tenant context before delegating to an application facade; an agent orchestrator that plans and executes the multi-agent workflow described in Section IV; and a data/operations plane consisting of PostgreSQL (application data, auth, guardrail audit, RAG chunks and embeddings via pgvector, evaluation and observability data) and Redis (session and async task handoff between the API and background workers).

B. Task Classification and Execution Planning

A QuestionClassifier assigns each question a task type (sql_query, chart, analytics, rag_explanation, verification, or file_data). An ExecutionPlanBuilder turns that into an ordered ExecutionPlan of agent steps grouped into three stages – sql, fanout, and verify – each step declaring its dependencies explicitly. A PlanExecutor runs the plan and records per-step timing, status, and errors independently of the final response, so a single agent's failure does not erase visibility into the steps that succeeded.

C. Governed SQL Model

SQL generation and SQL execution are deliberately separated by a non-negotiable guardrail step (Figure 2). A candidate SQL statement produced by the semantic layer is (1) parsed and restricted to a simple, single-statement SELECT with no DDL/DML; (2) checked against a

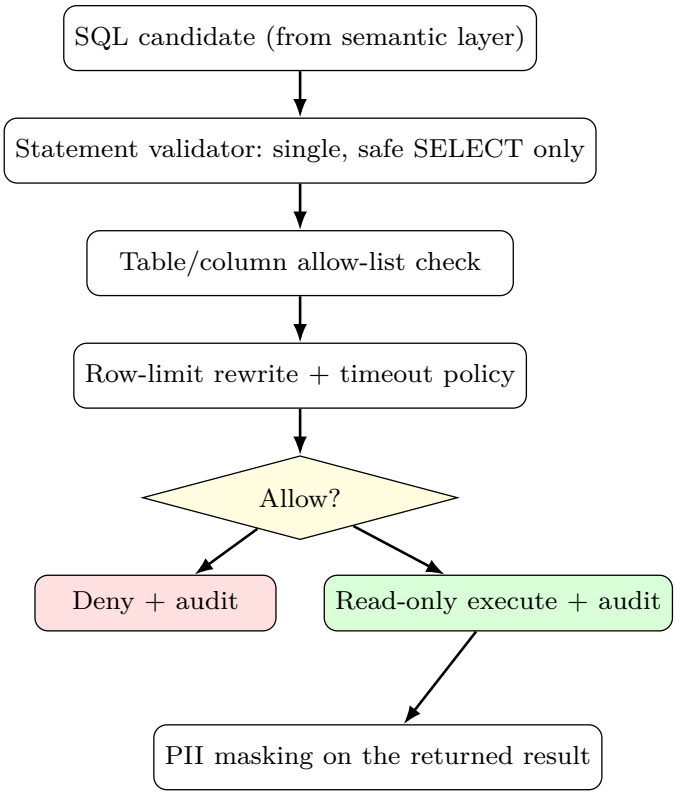

Fig. 2. SQL guardrail decision pipeline: nothing reaches the database without clearing every stage, and both outcomes are audited.

table/column allow-list; (3) rewritten to inject a row-count cap; (4) bounded by a query timeout policy; and only then (5) executed – and only against a separate, database-level read-only role, not the role the rest of the application uses. Every decision, allowed or denied, is written to an audit log keyed by the request's trace_id, and the result is masked for personally identifiable fields based on the caller's permissions before it is returned.

D. RAG Retrieval Model

Explanatory questions are answered from real business documents rather than from the model's own inference. Figure 3 shows the hybrid pipeline: documents are parsed, chunked, embedded, and stored in pgvector with owner/role metadata; a question is embedded and also scored lexically with BM25 over the already permission-filtered candidate set; the two scores are fused, narrowed to a top-$2K$ candidate set, optionally re-scored by a cross-encoder reranker, and assembled into a citation-grounded context that the LLM is restricted to answering from.

A golden dataset of real, schema-grounded questions with expected chunk IDs is scored separately with Hit Rate@K and Mean Reciprocal Rank (MRR), tracked as observability-only metrics until a production baseline exists to justify a release-gate threshold. This design was itself revised after a code-level audit found that three of its four documented stages were placeholders – a hash-bucket pseudo-embedding standing in for real vectors on one code path, token-overlap standing in for BM25, and a "rerank" step that resorted an already-computed score rather than running a second model pass. Each gap was written up and closed as a separate, numbered follow-up design before being implemented, and a subsequent review pass caught that the reload path was not actually consuming the backfilled pgvector vectors. That audit trail is kept in the repository rather than summarized away, because it is stronger evidence of engineering rigor than a document that only states intentions.

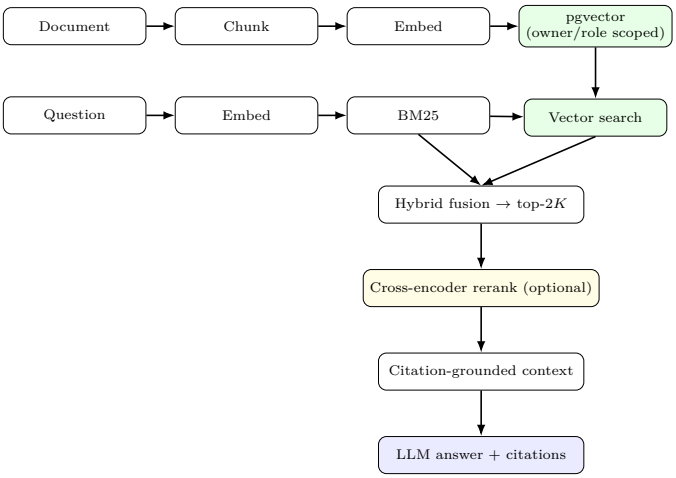

Fig. 3. Hybrid RAG pipeline: BM25 keyword scoring and real vector search are fused, permission-filtered, optionally reranked, and only then handed to the LLM as a bounded, citable context.

IV. Detailed Implementation

A. Backend Orchestration

The backend is a FastAPI application. An application facade coordinates domain workflows for the orchestrator, and the orchestrator wires together history, the SQL guardrail, and trace recording. Table I lists the agents involved and their single responsibility; each is independently unit-tested against its own contract rather than only end-to-end.

Confidence is aggregated rather than self-reported: a ConfidenceAggregator combines signals from the agents that actually ran (SQL success, RAG hit quality, verifier outcome), weighted per source, instead of trusting a single model's stated confidence. Long-running analytics/RAG jobs are handed off through a worker queue to a separate worker process rather than blocking the chat request.

B. Semantic Layer and NL2SQL

A semantic catalog constrains what the model is allowed to reference, so that a term such as "revenue" resolves to one agreed metric definition rather than whatever the model infers from a column name. A question parser extracts structural intent (metric, dimension, time window, filters) before generation, and a schema-drift detector flags

TABLE I
Agents and their responsibilities

Agent	Responsibility
SQL Agent	Turns a parsed question into a SQL candidate via the semantic layer and the sql_generation LLM route
SQL Guardrail	Validates, rewrites, and allow/deny-gates every SQL candidate before database access (Fig. 2)
Read-only Executor	Runs guardrail-approved SQL against the read-only Postgres role
Analytics Agent	Forecasting and anomaly detection over the query result
Visualization Agent	Produces a chart specification for chart-shaped questions
RAG Agent	Runs hybrid retrieval (Fig. 3) for explanatory questions
Federated Query Agent	Extends read-only querying to admin-approved business data sources
File Data Agent	Answers questions scoped to a user's uploaded file, respecting ownership/sharing
Verifier Agent	Checks the assembled answer against its sources before it is returned

TABLE II
Task-based LLM routing

Task type	Purpose
intent_classification	Cheap/fast routing of the question to a task type
sql_generation	Strong instruction-following model, grounded in the semantic catalog
answer_synthesis	Grounded only in bounded SQL rows and evidence snippets actually returned upstream
evidence_reasoning	Explanation answers, must cite evidence anchors when available

when the live database schema has moved away from what the catalog describes. The NL2SQL step never talks to the database directly; its only output is the SQL candidate that must clear the guardrail in Figure 2.

C. LLM Provider Gateway

Every model call goes through a single gateway that implements one provider-agnostic protocol (complete(request, route) → response), so timeouts, retries with backoff, and token/cost tracking are enforced in one place, and providers can be swapped without touching agent code. Table II shows the task-based routing model: the system deliberately makes several smaller model calls per question rather than one large one, so cheaper/faster models can be routed to classification while a stronger instruction-following model handles SQL generation.

The default provider is a deterministic, network-free mock – a deliberate choice, not a placeholder left in by accident, since it is what allows the automated test and evaluation suite described in Section VIII to run without API cost or dependency on a live model's uptime. OpenAIChatProvider is the first real adapter, implemented with the standard library's HTTP client rather than a heavy SDK dependency, defaulting to gpt-4o-mini as a cost/latency balance for a workload that issues several model calls per question. The same reasoning drives the default embedding model, text-embedding-3-small: 1536 dimensions at low per-token cost, adequate for chunk-level semantic search where recall matters more than maximum embedding fidelity. Because both the LLM and embedding clients are defined as protocols, adding another provider (e.g., Anthropic or Gemini) is an adapter, not a rewrite of the orchestrator.

D. Frontend

The frontend is built with React, Vite, and TypeScript, and is compiled to static assets served by nginx in the Docker Compose deployment. It includes a chat interface for the primary ChatBI flow and an admin console, reachable only with the appropriate admin:* permissions, that surfaces system health, LLM call/token/cost signals, SQL guardrail block counts, RAG hit rate, evaluation results, and release-gate status.

E. Authentication, RBAC, and Tenant Isolation

Authentication is implemented, not stubbed: organizations, users with hashed passwords and a token_version used for revocation, short-lived access tokens paired with longer-lived refresh sessions, and a role-audit-events table recording every permission change with its before/after state and actor. Admin-only endpoints check specific permission strings (e.g., admin:eval:read, admin:trace:read, admin:audit:read, admin:user:write) rather than a single coarse "is_admin" flag, so an organization can, for example, grant read access to observability data without granting write access to user roles. Tenant isolation scopes query history, RAG documents/embeddings, evaluation results, audit logs, and traces by organization; RAG's owner/role filtering is enforced in the SQL query itself, not applied after retrieval, which is what closes the class of cross-tenant leak that application-layer-only filtering would still be vulnerable to.

V. Reliability, Governance, and Security

A. Defense in Depth for SQL Safety

The guardrail described in Figure 2 is deliberately not the only safety layer. Even if the guardrail's own logic had a bug, the database connection it executes against carries a separate, database-level read-only role that has no write privileges at all; even if a write statement somehow slipped past both layers, the object-access allow-list still bounds which tables and columns are reachable. Every decision – allowed or denied – is written to an audit log keyed by the request's trace_id and queryable later by an admin.

B. Tenant Isolation as a Query-Level Guarantee

Rather than trusting an application-layer filter applied after retrieval, owner/role scoping for RAG documents is pushed into the SQL query against pgvector itself. This

distinction matters: a bug in application-layer filtering can silently leak a result before the filter runs, while a query-level constraint cannot return a row that violates it in the first place. A dedicated test asserts that one tenant cannot retrieve another tenant's chunks.

C. PII Masking and Audit Trail

Query results are masked for sensitive fields based on the caller's permissions after execution, and structured logs mask sensitive content before it is written. Every guardrail decision, role change, and admin access to trace/eval/audit data is recorded as an auditable event.

D. Provider Failure Handling

The LLM gateway enforces a timeout per model call, and retries transient failures with backoff; a failed SQL-generation call never falls through to execution, and answer-synthesis failures degrade to returning structured data rather than an invented summary. Every provider failure emits an observability event.

E. Evaluation Gates and Human Acceptance

An evaluation runner scores a suite of cases and produces run/score/failure records and a report; a release-gate decision combines automated tests, strict static type checks, and evaluation-quality thresholds into a single pass/fail signal. A human-acceptance step is required, but deliberately only after the machine gates already pass – business review can add judgment on top of a passing state, but it cannot override a failed one. We consider this an honest constraint on the release process, not an inconvenience: a system that lets a human override a failing test suite is not actually gated by that test suite.

VI. Deployment and DevOps

A. Local Deployment

Docker Compose brings up the React frontend, the FastAPI backend, an async worker, PostgreSQL 16, and Redis 7 with one command. In the default local mapping, the frontend is reachable at http://localhost:8080, the backend API at http://localhost:8000, PostgreSQL at localhost:5433, and Redis at localhost:6379.

B. Cloud Deployment

A Kubernetes manifest defines a namespace, Deployments and Services for each component, an Ingress, and a HorizontalPodAutoscaler for the backend (2–6 replicas, targeting 70% CPU). Its shape is checked by an architecture test, and – unlike a manifest that only exists on paper – it has been deployed to a real GKE Autopilot staging cluster and exercised under load; Section VII reports those results. This is staging-grade validation, not a hardened production profile: managed PostgreSQL and Redis, a real container registry, TLS on the ingress, a secrets manager used consistently across every environment, and prod-vs-staging manifest overlays are still the remaining steps, documented in the deployment runbook rather than claimed as complete.

TABLE III
GKE staging load benchmarks

Profile	Requests	Success	P95 (ms)
Steady-state, 0.8 RPS, 10 min	480	100% (480/480)	176.7
Burst, 50 req, concurrency 10	50	100% (50/50)	183.4

C. Continuous Integration

A GitHub Actions workflow runs the release-gate test and type-check bundle on every push, so the pass/fail signal described in Section VIII is enforced continuously rather than only checked manually before a demo. A second workflow builds and tests the container images on the cloud-deployment path and can trigger a staging rollout.

VII. Cloud Staging Validation

The Kubernetes deployment described above was not left unexercised. It was rolled out to a GKE Autopilot staging cluster behind a public HTTP endpoint, with the OpenAI API key injected from a Kubernetes Secret rather than committed to any manifest, and benchmarked under steady load, burst load, intentional overload, correctness regression, and a live pod-failure drill. The LLM provider for these runs was the deterministic mock (Section IV), so the results below are reproducible systems measurements – throughput, latency, recovery time, and governance behavior under load – not a measurement of real OpenAI answer quality or provider latency; validating the latter against a small, human-graded slice of the golden dataset is listed as future work.

A. Steady-State and Burst Load

Table III summarizes two load profiles against the live staging endpoint: a 10-minute steady-state run at a modest, sustainable rate, and a short 50-request burst at concurrency 10. Both completed with zero failed requests.

During the burst profile, a parallel 100-request guardrail sample showed a 100% dangerous-SQL detection rate, a 100% benign-SQL allow rate, and a guardrail P95 latency of 168.5 ms – the SQL guardrail's decision cost is a small, bounded fraction of the overall request latency, not a bottleneck.

B. Overload: Governance Under Sustained Excess Load

A separate 10-minute run deliberately targeted 20 RPS – well above the platform's configured rate-limit budget – to test what happens when demand exceeds the governed capacity, rather than only testing the happy path. Table IV shows the result: of 12,000 requests issued, 1,200 were accepted (HTTP 200) and 10,799 were rejected with HTTP 429; exactly one request failed with a transient HTTP 502. We read this as the intended behavior, not

TABLE IV
Overload run: 12,000 requests at a 20 RPS target

Status	Count	Share
200 (accepted)	1,200	10.0%
429 (rate-limited)	10,799	90.0%
502 (transient failure)	1	<0.01%

a failure: the platform fails closed under sustained excess load – rejecting the surplus at the rate limiter – rather than queuing unboundedly or letting every rejected request still consume a model/tool call downstream. The one HTTP 502 out of 12,000 requests is the more interesting number to watch as this test is repeated in the future.

C. Correctness Under a Live Endpoint

Two correctness suites were run against the live staging endpoint. A 3-case golden suite scored 66.7% (2/3), with golden_monthly_revenue the one failing case; a broader 12-case suite covering more phrasings of similar revenue and chart questions scored 100% (12/12). We report both rather than only the better number: the smaller golden suite is the stricter, curated signal, and a single named failing case is a concrete, reproducible regression target rather than noise to average away.

D. Failure Recovery

We deleted a running backend pod (backend-58486cf7b8-267zh) directly on the live cluster and timed recovery: the Deployment rolled a replacement pod to Running in 21 seconds, and a chat query issued immediately after returned HTTP 200. This exercises the actual Kubernetes self-healing path, not just its presence in the manifest.

E. Resource Footprint Under Load

After the 10-minute steady-state run, backend pods were still using only single-digit millicores of CPU and 115–120 MiB of memory each; the staging node stayed at 2% CPU and 5% memory; and the backend HPA held at its floor of 2/2 replicas, meaning this load profile never approached its scale-up threshold. This is expected given the mock LLM provider removes the dominant real-world cost (waiting on an external model call) from the request path, and should not be read as a claim about capacity headroom under a real provider's latency.

F. Token Efficiency of the Governed Path

Separately from the staging runs, a local estimator compared the governed path's model input size – schema, guardrail-bounded SQL rows, and top RAG evidence snippets – against a naive baseline that sends full table/document context, across six realistic, multi-evidence executive analytical questions (e.g., quantifying a revenue shortfall against forecast and root-causing it across campaign and support evidence and recommending mitigations, in one question). Table V shows a 98.01% average token reduction (54,449 to 1,085 tokens/question). The gap is wider than for a simple lookup question because the naive baseline's context size scales with how much source data a thorough analytical answer would otherwise require, while the governed path's cost stays bounded by the guardrail's row cap and the RAG retriever's top-k regardless of question complexity – so the token-efficiency advantage of governed retrieval grows, not shrinks, as questions get harder. These are estimator-based figures suitable for an engineering report, not provider-billed usage; reproducing them against the OpenAI provider's own usage fields is listed as future work.

TABLE V
Governed vs. naive-context token usage (6 executive questions)

Question (abbreviated)	Opt.	Naive	Reduct.
July revenue shortfall: root cause + Q4 mitigations	1096	54453	98.0%
Executive risk narrative: growth, campaign, SLA, churn	1124	54452	97.9%
Rank products by tickets, incidents, revenue impact	1108	54454	98.0%
Marketing concentration vs. revenue decline risk	1051	54444	98.1%
Analytics Hub timeout: isolated or recurring?	1066	54448	98.0%
Customer-retention brief across churn drivers	1063	54445	98.1%
Average	1085	54449	98.0%

VIII. Testing and Evaluation

A. Unit and Integration Tests

The test suite spans 195 files covering agent contracts, orchestration routing, guardrail rules, authentication/RBAC/tenant isolation, hybrid RAG retrieval and scoring, file upload and sharing, frontend state/props contracts, and Docker/Kubernetes architecture assertions.

B. Static Typing and Architecture Boundaries

The codebase is checked with pyright in strict mode across both source and test code, and dedicated architecture-boundary tests assert that each layer (API, application facade, orchestration, governance, agents) does not reach past its intended dependencies.

C. Baseline Benchmark Results

Table VI reports the platform's own local, mock-provider benchmark suite, generated by a reproducible benchmark script against a fixed commit. These numbers should be read as reproducible engineering evidence from a deterministic local environment – not as production load-test measurements, since they do not exercise a live LLM provider or a production-scale cluster.

TABLE VI
Local baseline benchmark (mock-provider, deterministic)

Category	Metric	Result
Accuracy	Overall evaluation score	1.0
Accuracy	SQL safety	1.0
Accuracy	Agent routing	1.0
Accuracy	RAG faithfulness	1.0
Detection	Dangerous SQL detection rate (1000 checks)	1.0
Detection	Benign SQL allow rate (1000 checks)	1.0
Performance	Guardrail P95 latency (ms)	0.0546
Concurrency	Chat API success rate (100 req, c=10)	1.0
Concurrency	Chat API P95 latency (ms)	70.03
Evaluation	50-case eval elapsed (ms)	0.39
Recovery	Retry recovered transient failure	True
Recovery	Circuit breaker half-open after cooldown	True
Timeout	Slow query denied by timeout policy	True
Hallucination	Unsupported-claim rate	0.1
Hallucination	Ungrounded-answer confidence	0.5
Observability	Trace lookup P95 (10,000 events, ms)	0.0011

D. Retrieval-Specific Evaluation

Beyond answer-level scoring, a golden dataset of real, schema-grounded questions with expected chunk IDs is used to compute Hit Rate@K and MRR for the retrieval stage specifically – distinguishing "did retrieval find the right evidence" from "did the final answer look plausible." This is tracked as an observability-only metric today, pending a production baseline before a release-gate threshold is set.

IX. Results and Discussion

The final version of InsightOps AI achieves the goals set at the start of the project. First, it supports governed natural-language querying, where SQL generation and SQL execution are separated by an auditable guardrail rather than trusted to a single model call. Second, it supports evidence- grounded explanation through a genuinely hybrid retrieval pipeline, not a vector-search-only shortcut, with citations and an explicit missing-evidence signal instead of invented answers. Third, it supports real security boundaries: authentication, permission-scoped RBAC, and query-level tenant isolation, rather than a single shared demo login. Fourth, it supports quality control through an evaluation runner and a release gate that a human can only accept after, never instead of, the machine gates passing. Fifth, it is deployable and staging-tested, not just architecturally scaffolded: a real GKE deployment served 100% of requests successfully under steady and burst load, rejected 90% of a deliberate 20 RPS overload with HTTP 429 rather than degrading uncontrolled, recovered a killed pod in 21 seconds, and its governed retrieval path measured 98% fewer tokens than a naive full-context baseline on realistic analytical questions (Section VII).

At the same time, several limitations remain. The GKE deployment is staging-grade, not a hardened managed-cloud production profile: it still needs managed PostgreSQL/Redis, TLS on the ingress, a secrets manager used consistently across every environment, and prod-vs-staging overlays. Distributed resilience beyond HTTP-level rate limiting – circuit breakers around the LLM gateway, dead-letter queues, and load shedding at the worker layer – is only partially present; OpenAI is currently the only real LLM/embedding provider implemented, though the provider protocols were designed to make adding another one an adapter rather than a rewrite; the demo/seed data is not yet large enough for realistic load testing at production scale; and the internal trace/log/metric stack has no external APM exporter yet. The one HTTP 502 observed in 12,000 overload requests, and the golden-suite's one failing correctness case, are also left as open, named items rather than rounded away. We view all of this as an honest list of remaining production-hardening work rather than gaps to be minimized in the description of the system.

Even with these limitations, we believe the project is a meaningful demonstration of what it takes to make an LLM-backed system trustworthy for enterprise use: the model's fluency is only half the problem, and the harder half – guardrails, auditability, tenant isolation, evaluation gates, and an honest record of gaps found and closed – is what this project spent most of its engineering effort on.

X. Individual Contribution

This was an individually completed project. The author was responsible for the full scope described in this report: the semantic layer and governed NL2SQL design, the SQL guardrail and its defense-in-depth model, multi-agent orchestration and execution planning, the hybrid RAG retrieval pipeline (including the audit that found and closed its placeholder stages), the LLM provider gateway and model-selection rationale, authentication/RBAC/tenant isolation, the evaluation runner and release gate, the observability stack and its durable-storage/mining loop, the frontend chat and admin console, the local Docker Compose and GKE staging deployments and their benchmark scripts, and the accompanying test suite and documentation.

XI. Conclusion and Future Work

InsightOps AI treats a natural-language business question as a governed, multi-agent workflow rather than a single trust-everything model call. The main lesson of this project is that making an LLM-backed system usable in an enterprise setting is primarily a governance and systems-engineering problem, not a prompting problem: the platform is only as trustworthy as the guardrail standing between the model and the database, the retrieval

pipeline's honesty about what it actually found, and the audit trail that lets someone explain, after the fact, why an answer was produced.

Future work includes: (1) adding distributed resilience patterns – circuit breakers, retry queues with dead-letter handling, bulkheads, and load shedding – around the LLM gateway and async workers; (2) adding a second real LLM/embedding provider behind the existing protocol abstraction to validate that the abstraction holds under a real second implementation, not just in design; (3) hardening the GKE staging deployment into a managed-cloud production profile – TLS on the ingress, a secrets manager used consistently across every environment, and a load profile heavy enough to actually push the existing HPA past its floor of 2 replicas, which the benchmarks in Section VII did not; (4) building a synthetic enterprise-scale dataset generator to validate the platform under realistic load; and (5) connecting the existing internal observability stack to an external APM/metrics backend for production-grade monitoring.

References

[1] FastAPI, "FastAPI Documentation." [Online]. Available: https://fastapi.tiangolo.com/

[2] pgvector, "pgvector: Open-source vector similarity search for Postgres." [Online]. Available: https://github.com/pgvector/pgvector

[3] Redis, "Redis Documentation." [Online]. Available: https://redis.io/docs/

[4] OpenAI, "OpenAI API Reference." [Online]. Available: https://platform.openai.com/docs/

[5] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009.

[6] Kubernetes, "Kubernetes Documentation." [Online]. Available: https://kubernetes.io/docs/

[7] React, "React Documentation." [Online]. Available: https://react.dev/